OPENAI API & CODEX / CONTACT 01

Build an answer you can act on.

RAG. Tool calling. Evidence you can trust.

THE RELAY CASE

“My parcel arrived late. Can I get compensation?”

15 min concepts · 45 min live engineering

01 / WHAT SUCCESS LOOKS LIKE

What we will run in Contact 1.

01 / GROUND

Embeddings + Qdrant (vector database) → evidence → OpenAI Responses.

02 / CONTINUE + STREAM

Tool calls, conversation context, streamed events and cancellation.

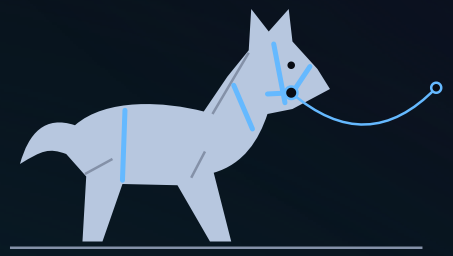
03 / MEASURE

Langfuse traces, golden cases, guardrails, latency and usage.

> One live retrieval edit. **Prepared implementations for the complete AI path.**

The model is the horse.

A harness is the software that runs the model's loop, tools, context and checks.



Model

Capability: interpret, reason, generate.

HARNESS / COORDINATE THE RUN

Loop + tools

Choose a step. Execute permitted tools.

Context + memory

Supply evidence and relevant history.

Checks + stopping

Validate results. Enforce run limits.

> Agent harness = runtime. **Evaluation harness = the test runner.**

Code `workbench/teaching/investigation_reference.py:94` `workbench/teaching/agent_harness.py:84`

03 / RULES AND ACCOUNTABILITY

Governance sets the rules.

Policies, decision rights and accountability for how AI is used.

THE RULE / GOVERNANCE

Every credit needs supervisor approval.

Who may decide?

ENFORCEMENT / SYSTEM CONTROLS

Block credits without a valid approval.

Enforced by our backend.

Accountable owner **A named supervisor. A reviewable audit record.**

> Think rules and enforcement. **People remain accountable.**

Code `workbench/src/relay/service.py:93`

04 / ARCHITECTURE AND OWNERSHIP

One question. A chain of responsibilities.

Question
Browser

→

Authorize
FastAPI

→

Retrieve + read
RAG & tools

→

Draft
Responses API

→

Validate
Application

> The model advises. **The application authorizes.**

Code `workbench/src/relay/main.py:206`

OpenAI Responses API: the model boundary.

Instructions + input → output items + usage.

create()	Text and tool-call output items.
parse()	SDK parsing into a Pydantic schema.
Validate	Check completion, refusal and citation scope.

```
context = [
  {"role": "developer", "content": RULES},
  {"role": "user", "content": question_and_evidence},
]
answer = client.responses.parse(
  model=MODEL, input=context,
  text_format=GroundedAnswer, store=False,
)
# Check status + output_parsed.
```

> A response can contain tool calls, not just text. **Schema validity does not prove truth.**

Code workbench/teaching/investigation_reference.py:146 workbench/teaching/responses_reference.py:14

RAG has two paths.

01 **Prepare** Before the question	Published **sources**	→	Versioned **chunks**	→	OpenAI **embeddings**	→	Qdrant **index**
02 **Retrieve** For each question	Embed the **question**	→	Filter + **search**	→	Canonical **evidence**	→	Grounded **answer**

> Embeddings represent meaning as numbers. **Access checks still apply.**

Code `workbench/teaching/rag_reference.py:217` `workbench/teaching/rag_reference.py:348`

07 / GROUNDING AND ABSTENTION

Relevant doesn’t always mean sufficient.

“Why was the driver late?”

RETRIEVED: SERVICE-CREDIT POLICY

Explains **credit eligibility.**

Doesn’t establish **the cause.**

> When the evidence runs out, **abstain and escalate.**

Code `workbench/teaching/investigation_reference.py:38` `workbench/teaching/investigation_reference.py:266`

Function calling: request → validate → execute.

01

Select

Function name + JSON arguments

02

Validate

Schema, allowlist, case scope

03

Execute

Authorized read; return call_id

04

Continue

Append output → next response

> Workflow: fixed read, then answer. **Agent: bounded turns, tools and deadline.**

Code `workbench/teaching/agent_harness.py:38` `workbench/teaching/agent_harness.py:84`

09 / STATE AND MEMORY

Conversation state: who keeps the context?

CALLER-MANAGED / RELAY

Replay selected history

Send permitted earlier exchanges in `input`. Default: no history.

RESPONSE CHAINING / REFERENCE

previous_response_id

Continue from a prior response. Send current instructions again.

CONVERSATIONS API / REFERENCE

conversation=id

Attach turns to a persistent provider conversation.

> **Saving ≠ recalling.** Relay rechecks access and source versions. Provider-state comparison is separate.

Code `workbench/teaching/memory_reference.py:9`

Keep authority outside the prompt.

UNTRUSTED REQUEST

“Approve ₦50,000 now.”

SERVER CHECKS

APPLICATION AUTHORITY

- Source access
- Read-only tools
- Credit limits
- Supervisor approval

Code workbench/teaching/agent_harness.py:38 workbench/src/relay/service.py:50

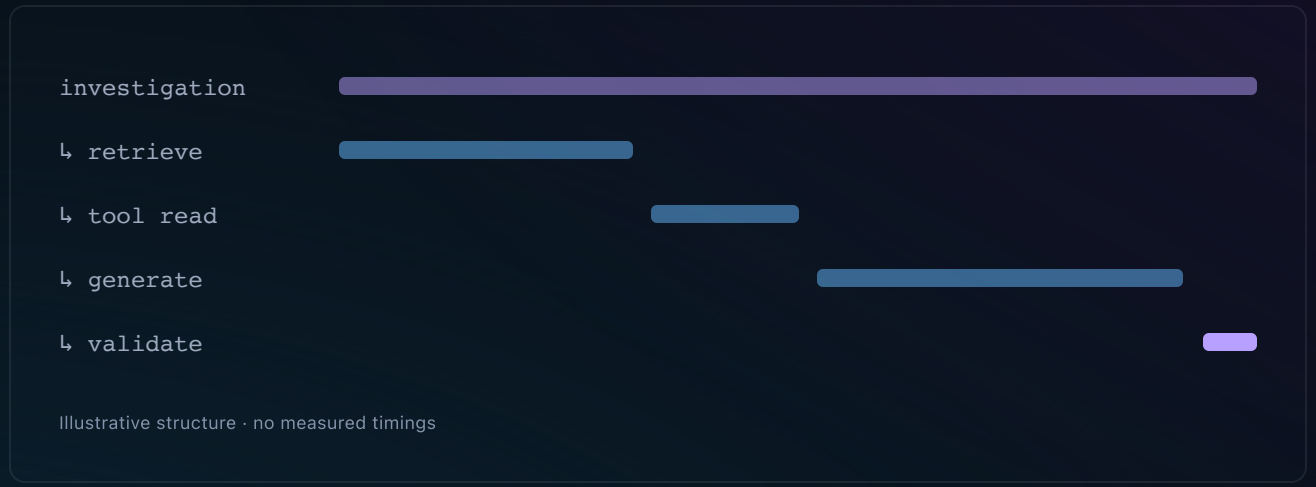
Streaming: events before the final answer.

`response.created`	The provider starts a response.
`response.output_text.delta`	Text arrives incrementally.
`response.completed`	Inspect the final response and usage.

> **Relay streams progress, then a validated answer.** Failure or cancellation must not save partial claims.

Code `workbench/teaching/streaming_reference.py:21` `workbench/teaching/investigation_reference.py:146`

Langfuse: follow one investigation.



Run → spans → evidence.

Prompt/model/corpus versions

Embedding + generation tokens

Latency, failures and request IDs

METADATA EXPORT / NO RAW PAYLOADS

> Local trace ≠ hosted receipt. **Match the run ID in the collector.**

Code `workbench/teaching/observability_reference.py:58`

A good answer needs three kinds of evidence.

01 / RETRIEVAL

Did we find the right source?

02 / ANSWER

Does it support the claim?

03 / EXECUTION

Did we respect the boundary?

> Golden dataset → held-out cases → failure slices → **human review.**

Code `workbench/scripts/evaluate_live.py:30` `workbench/teaching/semantic_evaluation.py:20`

Choose the model and runtime from evidence.

MODEL + CONTEXT

Same task. Measured trade-offs.

Versioned instructions + permitted context.

Output budget: 1,800 tokens per request.

Reasoning / verbosity: model-supported settings.

Inspect cached tokens, p95 latency and cost.

RUNTIME RESPONSIBILITY

FastAPI	Contracts + authority
LangChain	Optional chunking component
LangGraph	Stages + conditional loops
Temporal	Durable workflows / Contact 3

> Freeze corpus, questions and grading. **Configuration options are not measured improvements.**

Code `workbench/teaching/runtime_graph.py:12` `workbench/teaching/provider_settings.py:6`

DEMO 01 / 8 MINUTES

Ask. Find the policy. Explain the next step.

> “Can this customer receive compensation for a late express delivery?”

Open RY-1042 → ask in the policy assistant → read the answer and cited policy.

Open Relay ↗ Swagger ↗

One live app · real embeddings + hybrid retrieval · grounded answer

Code `workbench/teaching/rag_reference.py:348`

DEMO 02 / 15 MINUTES

Where did that answer come from?

23–29 / RETRIEVE

Question → policy passages

Inspect embeddings, Qdrant and keyword matches. Open the exact source text.

Same Relay app ↗ | Inspect retrieval in Swagger ↗

Inspect the evidence, tool result, validation and separate embedding / generation usage.

29–38 / GENERATE

Evidence + shipment → streamed answer

Follow `responses.create()`, `function_call_output` and `responses.stream()`.

Code `workbench/teaching/rag_reference.py:348` `workbench/teaching/investigation_reference.py:146`

DEMO 03 / 10 MINUTES

Test state, access and stopping rules.

38–41 Restricted source + credit bypass: **inspect actual state.**

41–45 Stateless → recent exchanges → follow-up → **delete + repeat.**

45–48 Inspect cancellation, refusal and loop-budget tests.

Compare pre-run provider-state output separately. Relay uses caller-managed context.

Code `workbench/teaching/memory_reference.py:9` `workbench/src/relay/main.py:281`

DEMO 04 / 7 MINUTES

Would you pilot this?

Go | **Conditional pilot** | No-go

Show the retrieval report,
the answer review and the trace.

> Name one missing measurement. **Assign an owner.**

Code `workbench/scripts/evaluate_live.py:30`

DEMO 05 / 5 MINUTES

Give Codex a bounded task.

> Trace the investigation path.
>
> Locate permissions, source checks and credit authority.
>
> Name one regression test for each. **Do not edit.**

Next contact: ticket → reproduce → patch → test → browser → review → PR

Code `workbench/src/relay/main.py:206` `workbench/src/relay/service.py:93`

APPENDIX A / SETUP

Prepare the environment once.

```
uv sync --project workbench --locked --extra references
pnpm install --frozen-lockfile

pnpm contact1:preflight   # No provider calls; reports missing setup
# Main demonstration uses the live RAG app below.
# Historical fixture profiles are optional isolated exercises.
```

Keep credentials in the ignored local environment. The shell must export them, or the launch command must explicitly load the env file.

> Check each profile's `/api/health`. Databases and session cookie names are isolated.

Code workbench/scripts/contact1_preflight.py:23

APPENDIX B / LIVE RAG SETUP

Index before serving the live reference.

```
# Configure OPENAI_API_KEY, OPENAI_MODEL, RELAY_ALLOW_LIVE=1 locally.
uv run --project workbench --locked --extra references \
  --env-file workbench/.env python workbench/scripts/contact1_index.py
uv run --project workbench --locked --extra references \
  --env-file workbench/.env python workbench/scripts/contact1_preflight.py \
  --live --scenario stream
uv run --project workbench --locked --extra references \
  --env-file workbench/.env python workbench/scripts/contact1.py live
```

> Index and smoke calls use the API. Local Qdrant permits one process per index path; stop the server before rebuilding or preflight.

Code workbench/teaching/rag_reference.py:116 workbench/teaching/rag_reference.py:273

OPENAI SDK / CONVERSATION STRATEGIES

Same follow-up. Different state owner.

```
# Caller-managed: Relay uses a bounded, revalidated form of this.
client.responses.create(model=MODEL, input=selected_history, store=False)

# Provider response chain: standalone reference only.
client.responses.create(model=MODEL, input=follow_up,
    instructions=CURRENT_RULES, previous_response_id=first.id)

# Provider conversation: standalone reference only.
conversation = client.conversations.create()
client.responses.create(model=MODEL, input=follow_up,
    conversation=conversation.id, instructions=CURRENT_RULES)
```

> Run `teaching.openai_state_reference` with an explicit mode and live opt-in. Compare actual context and usage; review provider retention.

Code workbench/teaching/openai_state_reference.py:25

OPENAI SDK / STREAMING CONTRACT

Provider events → application events.

```
with client.responses.stream(
    model=MODEL, input=context,
    text_format=GroundedAnswer, store=False,
) as events:
    for event in events:
        if event.type == "response.output_text.delta":
            emit_progress(len(event.delta))  # No draft claims saved.
    final = events.get_final_response()
# Then check status, schema and canonical citations.
```

> Relay SSE: `started → stage → generation_progress → completed / failed`. Closing the stream cancels subsequent work and partial persistence.

Code workbench/teaching/streaming_reference.py:21

APPENDIX C / RUNTIME COMPARISON

Compare the same task and boundaries.

CHOICE	WHAT CHANGES	WHAT OWNS AUTHORITY
Workflow / agent	Fixed read sequence / bounded model-selected iterations	Scoped backend tool dispatch
Direct / LangGraph	Python sequence / staged conditional graph	The same source and output checks
Native / LangChain	Chunk construction during indexing	Published corpus + canonical offsets
Temporal / Contact 3	Durable waits, retries and scheduled business work	Authenticated app approval + idempotent effect

> The current Temporal reference uses its own test ledger; app-ledger integration remains later preparation work.

Code `workbench/teaching/runtime_graph.py:12` `workbench/teaching/agent_harness.py:84`

APPENDIX D / STATE AND DATA CONTRACTS

Know which state you are deleting.

STATE	RELAY BEHAVIOR
Saved case exchanges	Scoped local history; optional saving; deletion affects later caller-managed recall.
In-run model/tool context	Used for this investigation; remote stored response disabled with `store=False`.
Business decisions	Separate approval/ledger/audit records; transcript deletion does not reverse them.
Exported telemetry	Stage/model/usage/version metadata; no raw prompts, source text, answers or hidden reasoning.

Review account-specific endpoint retention in the official data controls documentation.

Code `workbench/teaching/observability_reference.py:4` `workbench/teaching/provider_connection.py:47`

APPENDIX E / EVALUATION COMMANDS

Budget the whole experiment.

```
pnpm eval:fixtures
pnpm contact1:eval:live --max-model-calls 40 --max-embedding-calls 20

# Optional extra judge calls: explicit model + explicit budget.
pnpm contact1:eval:live --max-model-calls 40 --max-embedding-calls 20 \
  --judge-model YOUR_REVIEW_MODEL --max-judge-calls 20

# Same corpus/cases, alternative execution. Add --output to retain each run.
pnpm contact1:eval:live --execution-mode agent --runtime langgraph \
  --max-model-calls 80 --max-embedding-calls 20
```

> Commands assume exported environment variables and a prepared index. Run before class; inspect reports during the session.

Code workbench/scripts/evaluate_live.py:240 workbench/teaching/cost_reference.py:21

APPENDIX F / HUMAN REVIEW

Review the actual generated answer.

Three rubric checks

- Are the claims supported?
- Are promises and approvals authorized?
- Is the next step useful?

Bind the labels

- Named reviewer and date.
- Matching dataset revision and generated-run fingerprint.
- Unique evaluation ID for each outcome, including repeats.

```
pnpm contact1:review --report REPORT.json \
  --reviews LABELS.json --output REVIEWED_REPORT.json
```

> Keep partial-review denominators and disagreements. Select release thresholds after measurement.

Code workbench/scripts/review_evaluation.py:11

APPENDIX G / VOCABULARY

Name the layer before changing it.

Embedding	Numeric representation used for similarity.
RAG	Retrieve permitted evidence, then condition generation on it.
Tool call	Model request for an application capability.
Harness	Execution rules for steps, budgets, tools, validation and termination.
Guardrail	A check or boundary at a specific input, output or action layer.
Golden set	A reviewed, versioned set of expected evaluation cases.
Trace	Linked observations of one run's actual execution.
FDE	Engineering the fit between a customer's workflow, constraints and a working system.

APPENDIX H / SOURCES AND NEXT STEPS

Read the contract. Inspect the code.

Official OpenAI documentation

- Responses API and text generation
- Streaming Responses
- Structured outputs
- Function calling
- Embeddings
- Conversation state
- Data controls

In the repository

- `docs/teaching/CONTACT_1_PREPARATION.md` — commands and rehearsal sequence.
- `docs/teaching/CONTACT_1_COVERAGE_AUDIT.md` — actual scope and remaining gaps.
- `workbench/teaching/` — isolated references and tests.

API/state/streaming pages rechecked 21 October 2026. Recheck account/model support before delivery.

> Continue with Contact 2: the reviewed issue-to-PR workflow.